```
class GuestsCleanupJob < ActiveJob::Base
  queue_as :default

  before_enqueue do |job|
    # Do something with the job instance
  end

  around_perform do |job, block|
    # Do something before perform
    block.call
    # Do something after perform
  end

  def perform
    # Do something later
  end
end
```

7 Action Mailer

One of the most common jobs in a modern web application is sending emails outside of the request-response cycle, so the user doesn't have to wait on it. Active Job is integrated with Action Mailer so you can easily send emails asynchronously:

```
# If you want to send the email now use #deliver_now
UserMailer.welcome(@user).deliver_now

# If you want to send the email through Active Job use #deliver_later
UserMailer.welcome(@user).deliver_later
```

8 GlobalID

Active Job supports GlobalID for parameters. This makes it possible to pass live Active Record objects to your job instead of class/id pairs, which you then have to manually deserialize. Before, jobs would look like this:

```
class TrashableCleanupJob < ActiveJob::Base
  def perform(trashable_class, trashable_id, depth)
    trashable = trashable_class.constantize.find(trashable_id)
    trashable.cleanup(depth)
  end
end
```

Now you can simply do:

```
class TrashableCleanupJob < ActiveJob::Base
  def perform(trashable, depth)
    trashable.cleanup(depth)
  end
end
```

This works with any class that mixes in `GlobalID::Identification`, which by default has been mixed into Active Record classes.

9 Exceptions

Active Job provides a way to catch exceptions raised during the execution of the job:

```
class GuestsCleanupJob < ActiveJob::Base
  queue_as :default
```